PO Notes New Persistency

Architecture Comparison & Recommendation Guide

Notes Reuse RAP | Sales SDD Own Table | ZPO_LONGTEXT | SAPScript Handler | SDM

SAP S/4HANA Procurement | EKKO / EKPO | CE2702 / CE2708 Planning | 2026-06-30

What This Document Covers

Section	Title	Key Content
1	Three Persistency Approaches	Generic table, Own table, RAP Notes Reuse — table structures
2	Sales SDD Table Design	SDTP_TEXT fields, ITF conversion, auth encoding
3	ZPO_LONGTEXT Current Design	Current Lift & Shift table structure
4	Full Comparison Table	All aspects side by side across all 3 approaches
5	Recommendation for PO	Which approach and why — proposed EKKT_TEXT design
6	SAPScript Handler Class	What it is, how it works, analogy
7	SDM — Silent Data Migration	What SDM does, step by step, how it connects to handler
8	Handler vs SDM — Key Difference	Summary table and one-line definitions

Context: CE2702/CE2708 sync-up meeting (2026-06-25) identified 3 persistency options for PO notes migration. Sales team completed their migration in CE2608 as the reference implementation. The critical open question — can application-specific own table work with Notes Reuse RAP — is pending with the Notes Reuse team.

1 Three Persistency Approaches — Table Structures

Generic Basis table vs Application-specific own table vs Current ZPO_LONGTEXT

Approach 1 — Generic Table: SGBT_NTCEONT (Basis/BCSRV Team)

MANDT	CLNT 3	Client
OBJECT_TYPE	CHAR 10	Business Object Type — generic, not EKKO/EKPO specific
OBJECT_ID	CHAR 70	Generic document ID — CHAR70 (same limitation as STXH TDNAME)
NOTE_ID	CHAR 4	Note/Text type
LANGUAGE	LANG 1	Language key
NOTE_CONTENT	STRG —	Long string — plain text content
CREATED_BY	CHAR 12	Created by user
CREATED_AT	DATS 8	Created date
CHANGED_BY	CHAR 12	Last changed by
CHANGED_AT	DATS 8	Last changed date

Owner: BCSRV component (Basis team). Already used by Supplier Confirmation (new feature, no migration needed). Notes Reuse RAP Business Object is built on this table. CDS views and C1 contract views owned by Basis — not by PO team. CHAR70 OBJECT_ID = same generic limitation as STXH TDNAME.

Approach 2 — Application-Specific Own Table: SDTP_TEXT (Sales Reference)

MANDT	CLNT 3	Client
SD_DOCUMENT_ID	CHAR 10	Properly typed Sales Order number (= VBELN) — NOT generic CHAR70
SD_DOCUMENT_ITEM	NUMC 6	Item number (= POSNR) — properly typed
TEXT_OBJECT	CHAR 10	VBBK (header) or VBBP (item)
TEXT_ID	CHAR 4	Text type (0001, 0002 etc.)
LANGUAGE	LANG 1	Language key
TEXT_CONTENT	STRG —	Plain text string — converted from ITF format via CONVERT_ITF_TO_STREAM_TEXT
REF_TEXT_OBJECT	CHAR 10	Reference chain — source object
REF_TEXT_ID	CHAR 4	Reference chain — source text ID
REF_TEXT_NAME	CHAR 70	Reference chain — source text name
CREATED_BY/AT	CHAR+DATS	Administrative fields
CHANGED_BY/AT	CHAR+DATS	Administrative fields
AUTH_ENCODED	CHAR ~20	Auth fields encoded via TDTITLE: DocCategory+DocType+SalesOrg+DistCh+Division
SDM_VERSION	NUMC 2	Migration status — in root table VBAK (not in text table itself)

Owner: SD LOB team. Handler class registered in TTXOB for VBBK/VBBP. SAP Script X team explicitly recommended this approach for all LOBs. Sales completed PoC in CE2608. SDM migrated 2.67M records in CCW/720 in 3.5 hours. ITF format converted to plain text string during migration.

Approach 3 — Current Lift & Shift Table: ZPO_LONGTEXT		
MANDT	CLNT 3	Client
EBELN	CHAR 10	PO Number — properly typed ✓
EBELP	NUMC 5	Item number — 00000 = header, 00010 = item 10
TDOBJECT	CHAR 10	EKKO (header) or EKPO (item)
TDID	CHAR 4	F01, F02, F09 etc.
TDSPRAS	LANG 1	Language key: D confirmed for this system
LINE_COUNTER	NUMC 5	Line sequence 00001, 00002... — ORDER BY field
TDFORMAT	CHAR 2	Format indicator: * = normal text line
TDLINE	CHAR 132	PLAIN TEXT content — one 132-char line per row (NOT a string)
MIGRATED_AT	DATS 8	Date row was written
MIGRATED_BY	CHAR 12	User who wrote this row (sy-uname)
SOURCE	CHAR 1	D = Dual-write by BAdI, M = Migration report

Integration: BAdI ME_PROCESS_PO_CUST fires AFTER SAVE_TEXT. No SDM_VERSION tracking, no handler class, no automatic read/write switching. Always dual-write (BAdI always fires). PoC/Lift & Shift approach only. EKPO TDNAME confirmed: NO space between EBELN and EBELP on this system (e.g. 450007774400010).

2 Sales SDD — ITF Conversion and Auth Encoding

How text format is converted and authorization info is transferred

ITF Format Conversion (Critical for Migration)

SAPscript stores text in ITF (Interchange Text Format) — a proprietary SAP format. The new persistency stores PLAIN TEXT strings. Conversion happens during migration:

```
" ITF table (TLINE rows) → Plain text string

CALL FUNCTION 'CONVERT_ITF_TO_STREAM_TEXT'

  EXPORTING i_type = 'TEXT'  TABLES i_text = lt_tline_table

  IMPORTING e_text = lv_plain_string.



" Plain text string → ITF table (for reverse – e.g. display in editor)

" Step 1: Split string into 132-char chunks

" Step 2: Convert

CALL FUNCTION 'CONVERT_STREAM_TO_ITF_TEXT'

  EXPORTING i_type = 'TEXT'  TABLES i_text = lt_stream_table

  IMPORTING e_text = lt_tline_table.
```

Authorization Encoding via TDTITLE Field

Auth fields are application-specific — not part of the SAPscript framework interface. Confirmed by text framework colleague Rene Zink: THEAD-TDTITLE field can be reused to pass application-specific data to the handler class.

Auth Field	From	Example Value
SD Document Category	VBAK-VBTYP	C
Sales Document Type	VBAK-AUART	TA
Sales Organization	VBAK-VKORG	1000
Distribution Channel	VBAK-VTWEG	10
Division	VBAK-SPART	00

```
" Concatenated value in TDTITLE (Option 1 – preferred):

" SD Doc Category + Sales Doc Type + Sales Org + Dist Ch + Division

" Example: 'C   TA  10001000'  (values padded with spaces)



" PO equivalent would be:

" Purch Org + Company Code + Document Type + ... encoded similarly
```

3 Full Comparison — All Three Approaches

Every architectural aspect side by side

Aspect	RAP Generic Table (SGBT_NTCEONT)	Sales SDD Own Table (SDTP_TEXT)	ZPO_LONGTEXT (Current)
Table owner	Basis / BCSRV	SD LOB team	Lift & Shift / PO team
Table name	SGBT_NTCEONT	SDTP_TEXT	ZPO_LONGTEXT
Key design	CHAR70 OBJECT_ID (generic)	Typed VBELN + POSNR ✓	Typed EBELN + EBELP ✓
Text storage	Long string (plain)	Long string (ITF→plain)	One row per 132-char line
Integration method	RAP Business Subject in BO model	Handler class in TTXOB	BAdI ME_PROCESS_PO_CUST
SAVE_TEXT routing	Not used — RAP APIs only	Auto via framework	Separate hook after SAVE_TEXT
Migration mechanism	N/A — new feature only	SDM CL_SDM_PACKAGE_MIGRATION	Manual report — no SDM
Dual-write control	N/A	Auto via IS_MIGRATION_FINISHED	Always on — BAdI always fires
Read switching	N/A	Auto at SDM completion	No cutover logic — always reads ZPO
CDS views	Built by Basis (C1 released)	Built by SD team on SDTP_TEXT	Not built yet
Analytics/BW	Via Basis CDS views	Via SD CDS views	Not supported
Auth fields	Not possible	YES — via TDTITLE encoding	Not stored
Text references	Not supported (confirmed)	Stored in REF_ fields	Not handled
App-specific attributes	NOT possible	YES — any field	YES — custom table
SDM_VERSION in root	Not required	Required in VBAK	Not required
Notes Reuse RAP compat.	YES — designed for it	UNKNOWN (open question)	UNKNOWN (open question)
SAP Script X recommendation	Not mentioned	YES — recommended	Aligned with this
Production ready	Yes — Supplier Confirmation	Yes — Sales CE2608 PoC	PoC only

4 Recommendation for Standard PO (EKKO / EKPO)

Sales SDD own table approach — pending Notes Reuse compatibility confirmation

RECOMMENDED: Sales SDD approach — Application-specific own table + SAPScript Handler Class Framework + SDM (Silent Data Migration) PENDING: Confirm with Notes Reuse team that own table is compatible with Notes Reuse RAP BO

Why NOT Generic Table (SGBT_NTCEONT) for PO

Concern	Detail
No migration path for existing texts	SGBT_NTCEONT is for NEW business objects only. PO has millions of existing texts in STXH/STXL needing migration.
CDS views owned by Basis	PO team cannot add EBELN-specific joins or PO analytics without Basis involvement.
CHAR70 OBJECT_ID	Still generic — same JOIN difficulty as STXH TDNAME. No EBELN typed key.
No PO-specific fields	Cannot add Purchase Org, Document Type for authorization or reporting.
References not supported	Direct text references confirmed NOT supported in new persistency (SDD confirmed).

Why Sales SDD Own Table IS Recommended for PO

Benefit	PO-Specific Reasoning
Typed keys EBELN + EBELP	Direct SQL joins with EKKO, EKPO — no CHAR70 parsing needed
PO team owns the table	Can add EKKO-specific fields: Purchasing Org, Company Code, Doc Type for auth
SAP Script X recommended	Explicit guideline to all LOBs — PO follows same path as Sales
SDM framework proven	Sales completed this in CE2608 — reusable pattern for PO in CE2708
Automatic read/write switching	IS_MIGRATION_FINISHED handles cutover — no manual steps needed
AI and BDC ready	Plain text string in single field — AI can process directly
CDS views on PO terms	Analytics CDS can join with I_PurchaseOrderBasic, I_PurchaseOrderItem directly

Proposed PO Table: EKKT_TEXT

EKKT_TEXT — Recommended PO New Text Table		
MANDT	CLNT 3	Client
EBELN	CHAR 10	PO Number — properly typed (NOT generic CHAR70) ✓
EBELP	NUMC 5	Item — 00000 = header, 00010 = item 10 ✓
TEXT_OBJECT	CHAR 10	EKKO (header) or EKPO (item)
TEXT_ID	CHAR 4	F01, F02, F09 etc.
LANGUAGE	LANG 1	Language key: D for this system
TEXT_CONTENT	STRG —	Plain text string — ITF converted via CONVERT_ITF_TO_STREAM_TEXT
REF_TEXT_OBJ	CHAR 10	Reference chain — source object
REF_TEXT_ID	CHAR 4	Reference chain — source text ID
REF_TEXT_NAME	CHAR 70	Reference chain — source text name
CREATED_BY/AT	CHAR+DATS	Administrative — created by/date
CHANGED_BY/AT	CHAR+DATS	Administrative — changed by/date
AUTH_PORG	CHAR 4	Purchase Organization — explicit auth field (better than encoding)
AUTH_BUKRS	CHAR 4	Company Code — explicit auth field
AUTH_BSART	CHAR 4	Document Type — explicit auth field

KEY DIFFERENCE from ZPO_LONGTEXT: TEXT_CONTENT = full plain text STRING (not one row per 132 chars — no LINE_COUNTER needed) Auth fields stored EXPLICITLY (not encoded via TDTITLE like Sales) Reference fields stored No TDFORMAT / TDLINE / LINE_COUNTER columns

Migration Sequence

Release	Task
CE2702 — PoC	Clarify Notes Reuse compatibility. Design EKKT_TEXT. Implement ZCL_PO_RSTXT_PERSISTENCE handler class. Register in TTXOB for EKKO and EKPO. Add SDM_VERSION to EKKO. Implement ZCL_SDM_EKKO_TEXT_MIGRATION. POC: write to both STXH and EKKT_TEXT via modernized OData V4 API.
CE2708 — Full	SDM runs in background — migrates all historic PO texts. IS_MIGRATION_FINISHED → TRUE → read switches to EKKT_TEXT. Build CDS views. Plug notes into PO OData V4 API. Regression test for all EKKO/EKPO business objects.

5 SAPScript Handler Class Framework

Runtime plug-in that intercepts SAVE_TEXT and READ_TEXT — routes to new table

ANALOGY: Think of the Handler Class as a TRAFFIC CONTROLLER standing at the SAVE_TEXT/READ_TEXT junction. It redirects traffic to the new table. Without it, all traffic goes to STXH/STXL as before.

What It Is

A runtime plug-in registered in table TTXOB that intercepts every SAVE_TEXT and READ_TEXT call and routes them to your new table instead of STXH/STXL. It is a class that inherits from CL_RSTXT_PERSISTENCE_FRAMEWORK.

```
WITHOUT Handler Class (before registration):

  SAVE_TEXT → always writes to STXH/STXL

  READ_TEXT → always reads from STXH/STXL


WITH Handler Class registered in TTXOB for EKKO and EKPO:

  SAVE_TEXT → SAP framework checks TTXOB → calls ZCL_PO_RSTXT_PERSISTENCE

               → your CREATE or CHANGE method runs

               → writes to EKKT_TEXT

               → AND writes to STXH/STXL if IS_MIGRATION_FINISHED = FALSE


  READ_TEXT → SAP framework calls ZCL_PO_RSTXT_PERSISTENCE→READ

               → if IS_MIGRATION_FINISHED = FALSE → reads STXH/STXL

               → if IS_MIGRATION_FINISHED = TRUE  → reads EKKT_TEXT
```

5 Methods to Implement

Method	When Called	What It Does
CREATE	Every SAVE_TEXT creating new text	Write new text to EKKT_TEXT (+ STXH during migration)
CHANGE	Every SAVE_TEXT updating text	Update text in EKKT_TEXT (+ STXH during migration)
DELETE	Every DELETE_TEXT call	Remove text from EKKT_TEXT (+ STXH during migration)
READ	Every READ_TEXT call	Read from EKKT_TEXT (new) or STXH/STXL (old) based on IS_MIGRATION_FINISHED
READ_HEADERS_VIA_RANGES	Every SELECT_TEXT call	Read text headers from EKKT_TEXT or STXH

Registration in TTXOB

```
Table TTXOB — Text Object definition:

  TDOBJECT = 'EKKO'  → HANDLER_CLASS = 'ZCL_PO_RSTXT_PERSISTENCE'

  TDOBJECT = 'EKPO'  → HANDLER_CLASS = 'ZCL_PO_RSTXT_PERSISTENCE'


After registration — ALL applications using SAVE_TEXT/READ_TEXT for EKKO/EKPO
```

```
automatically route through your handler class — no code changes in ME21N/ME22N needed.


Alternative: Register at TDID level in TTXID (per text type) instead of all TDIDs.
```

6 SDM — Silent Data Migration

Background batch job that moves EXISTING historic texts from STXH/STXL to new table

ANALOGY: Think of SDM as REMOVAL MEN moving all the furniture from your old house to your new house while you are still living there. The Handler Class is the new house address label (new deliveries go to new house). But someone still has to move everything already in the old house. That is what SDM does.

What Problem SDM Solves

```
After Handler Class is registered:


  NEW texts (created after registration):
    → Handler Class routes them to EKKT_TEXT immediately ✓

  EXISTING historic texts (created BEFORE registration):
    → Still ONLY in STXH/STXL ✗
    → PO 4500001234 from 2020 — its text is ONLY in STXH/STXL

SDM migrates all the HISTORIC data in background — silently.
```

SDM Step by Step — What It Actually Does

Step	Action	Detail
1	Select package	SELECT ebeln FROM ekko WHERE sdm_version < target UP TO 30,000 ROWS
2	Read old texts	CALL FUNCTION 'SELECT_TEXT' → get text headers CALL FUNCTION 'READ_TEXT_TABLE' → mass read text content (decompresses STXL)
3	Convert format	CALL FUNCTION 'CONVERT_ITF_TO_STREAM_TEXT' → ITF table → plain text string
4	Delete existing	DELETE FROM ekkt_text WHERE ebeln IN @lt_package — ensures restartability
5	Insert new	INSERT ekkt_text FROM TABLE lt_new_texts — plain text rows in new table
6	Update status	UPDATE ekko SET sdm_version = target WHERE ebeln IN @lt_package
7	Repeat	Process next package until all POs have sdm_version = target

Performance Numbers from Sales (Reference)

System	VBAK Records	Text Records	Package Size	Duration
DDCI (test)	40,853	340	30,000	Fast
CCW/720 (perf)	13.9 million	2.67 million	30,000	3.5 hours

Open SQL limit: max 32,767 entries in WHERE range table → package size set to 32,000. Internal packaging within SELECT_TEXT calls every 30,000 documents. Old STXH/STXL records are NOT deleted during SDM — kept as safety net for restart. PO estimate: if similar volume to Sales (~13M POs) → ~3.5 hours SDM duration.

IS_MIGRATION_FINISHED — The Bridge Between Handler and SDM

```
IS_MIGRATION_FINISHED = FALSE  (SDM still running — called in handler class):
```

```
  Handler CREATE/CHANGE → writes to STXH/STXL AND EKKT_TEXT (dual-write)
  Handler READ          → reads from STXH/STXL (complete data — SDM still running)

IS_MIGRATION_FINISHED = TRUE   (SDM complete — all data in new table):
  Handler CREATE/CHANGE → writes to EKKT_TEXT ONLY
  Handler READ          → reads from EKKT_TEXT (new — complete)

Implementation:
  METHOD is_migration_finished.
    rv_finished = cl_sdm_proc_status_api=>is_sdm_finished(
      i_sdm_name = 'ZCL_SDM_EKKO_TEXT_MIGRATION' ).
  ENDMETHOD.

Result buffered for entire dialog session — avoids repeated SELECT statements.
```

7 Handler Class vs SDM — Key Differences

They are two completely different mechanisms that work together

They Are NOT the Same Thing: Handler Class = RUNTIME routing (where does NEW text go?) SDM = BACKGROUND migration (how do EXISTING historic texts get moved?)

Aspect	SAPScript Handler Class Framework	SDM — Silent Data Migration
What it is	Runtime plug-in registered in TTXOB	Background batch migration job
What it solves	Routes NEW SAVE_TEXT/READ_TEXT to new table	Moves EXISTING historic data to new table
When it runs	Every time SAVE_TEXT or READ_TEXT is called	Once in background — packages of 30,000
How activated	Register handler class in TTXOB.HANDLER_CLASS	Execute SDM job from SDM cockpit
Duration	Instant — runs per transaction	Hours (3.5h for 13M records)
Who drives it	SAP Script framework — automatic	SDM cockpit — background job
Controls read switch	YES — via IS_MIGRATION_FINISHED	NO — it just migrates data
SAP class to inherit	CL_RSTXT_PERSISTENCE_FRAMEWORK	CL_SDM_PACKAGE_MIGRATION
Sales class name	Part of SD text persistency class	CL_SDM_SD_VBAK_TEXT_MIGRATION
PO equivalent to build	ZCL_PO_RSTXT_PERSISTENCE	ZCL_SDM_EKKO_TEXT_MIGRATION

Full Timeline — How They Work Together

```
BEFORE any change:

  STXH/STXL has all PO texts (old house – all furniture here)

  EKKT_TEXT is empty (new house – empty)


STEP 1 – Register Handler Class in TTXOB for EKKO/EKPO:

  New texts → go to EKKT_TEXT (AND STXH/STXL while SDM running)

  Historic texts → still only in STXH/STXL

  IS_MIGRATION_FINISHED = FALSE → reads still from STXH/STXL


STEP 2 – SDM runs in background (hours/days):

  Package by package: reads STXH/STXL → converts → inserts EKKT_TEXT

  Updates SDM_VERSION in EKKO for each completed package

  System continues working normally throughout


STEP 3 – SDM completes (all SDM_VERSION updated):

  IS_MIGRATION_FINISHED = TRUE

  Handler Class now reads from EKKT_TEXT
```

```
  Handler Class now writes to EKKT_TEXT only

  STXH/STXL = frozen/legacy (kept as safety net)


STEP 4 — After validation period:

  STXH/STXL rows for PO texts cleaned up (archiving)

  EKKT_TEXT is the single source of truth ✓
```

In One Sentence Each

Mechanism	One-Line Definition
SAPScript Handler Class Framework	A plug-in that intercepts every SAVE_TEXT and READ_TEXT call and routes it to your new table — effective immediately for all new transactions.
SDM (Silent Data Migration)	A background job that moves all the existing historic text data from STXH/STXL to your new table silently while the system keeps running normally.

8 SDTP_TEXT — REF Fields vs TEXT Fields Explained

What REF_TEXT_OBJECT, REF_TEXT_ID, REF_TEXT_NAME store and when they are populated

Identity Fields vs Reference Pointer Fields

Field	Type	Populated When	Contains
TEXT_OBJECT	Identity	Always	Which SAP business object this text belongs to — e.g. VBBK or EKKO
TEXT_ID	Identity	Always	Which type of text within that object — e.g. F01, F02, F09
TEXT_CONTENT	Content	Only when text is a COPY (own text)	The actual plain text string the user typed — empty if reference
REF_TEXT_OBJECT	Reference pointer	Only when text is a REFERENCE	Object to look up for real content — e.g. LFA1, MATERIAL, EINE, EKKO
REF_TEXT_ID	Reference pointer	Only when text is a REFERENCE	Text type to look up in the referenced object — e.g. 0002
REF_TEXT_NAME	Reference pointer	Only when text is a REFERENCE	Document/record ID to look up — e.g. vendor number 0000001000

Two Types of Rows in SDTP_TEXT

```
TYPE A — Copy (own text — user typed it directly):

  SD_DOCUMENT_ID  = 230001

  TEXT_OBJECT     = VBBK

  TEXT_ID         = 0001

  TEXT_CONTENT    = 'Deliver before Christmas'  ← HAS CONTENT

  REF_TEXT_OBJECT = ''   ← EMPTY — not a reference

  REF_TEXT_ID     = ''   ← EMPTY

  REF_TEXT_NAME   = ''   ← EMPTY
```

```
TYPE B — Reference (pointer to another document's text):

  SD_DOCUMENT_ID  = 230002

  TEXT_OBJECT     = VBBK

  TEXT_ID         = 0001

  TEXT_CONTENT    = ''   ← EMPTY — no own content

  REF_TEXT_OBJECT = 'KNA1'        ← Points to: Customer master

  REF_TEXT_ID     = '0001'         ← Points to: text type 0001 in KNA1

  REF_TEXT_NAME   = '0000012345'   ← Points to: Customer ID


Reading Sales Order 230002 header note:

  → TEXT_CONTENT is empty → check REF fields

  → Go read KNA1/0001/0000012345 to get the actual text
```

Where REF Fields Are Populated From

Source	How REF Fields Are Set
During SDM Migration (historic documents)	Read from STXH fields: TDREF='X' → reference. TDREFOBJ→REF_TEXT_OBJECT, TDREFNAME→REF_TEXT_NAME, TDREFID→REF_TEXT_ID. If TDREF=' ' (copy) → TEXT_CONTENT set, REF fields empty.
Runtime — Handler CREATE/CHANGE	SAVE_TEXT passes THEAD structure. If THEAD-TDREF='X' → store REF fields, TEXT_CONTENT=''. If TDREF=' ' → convert ITF→string, store TEXT_CONTENT, clear REF fields.
User edits a referenced text	CHANGE method detects content is being written → clears REF_TEXT_OBJECT, REF_TEXT_ID, REF_TEXT_NAME → stores new content in TEXT_CONTENT. Reference is permanently broken.

The Reference Chain Problem

```
Reference chain example (can be N levels deep):


  KNA1 (Customer master)

    text 0001: 'Standard delivery instructions'

       ■  REF_TEXT_OBJECT=KNA1, NAME=0000012345

       ■■■ VBBK (Quotation 100001) references KNA1

              ■  REF_TEXT_OBJECT=VBBK, NAME=100001

              ■■■ VBBK (Sales Order 230001) references Quotation

                     ■  REF_TEXT_OBJECT=VBBK, NAME=230001

                     ■■■ VBBK (Delivery 800001) references Sales Order


To read Delivery 800001 text:

  Step 1: Read SDTP_TEXT Delivery 800001 → empty → follow REF to Sales Order 230001

  Step 2: Read SDTP_TEXT Sales Order 230001 → empty → follow REF to Quotation 100001
```

```
  Step 3: Read SDTP_TEXT Quotation 100001 → empty → follow REF to KNA1

  Step 4: Read KNA1 text table → 'Standard delivery instructions' ✓


CANNOT be resolved via CDS JOIN — chain depth is unknown.

Only 1-level self-join supported (like CRM solution).

Cross-object refs (EKKO→LFA1, EKKO→MATERIAL) impossible without missing CDS views.
```

■ ~2% of text records in customer systems have cross-object references. 98% are copy records or same-object references — these are manageable. Cross-object references remain a known limitation — KBA note to customers required.

9 PO Reference Notes — 6 Scenarios

When and why reference texts are created for EKKO and EKPO

Configuration Control — VOTXN and SPRO Paths Clarified

■ Two separate text configurations exist for PO — they serve DIFFERENT purposes: 1. VOTXN / Text Determination = controls copy/reference rules when PO is CREATED 2. SPRO → Messages → Texts for Messages → Define Texts for PO = controls which texts are PRINTED on PO output form sent to vendor For REF fields and reference behaviour → use VOTXN (path 1).

Path 1 — Text Determination (Copy/Reference Rules): VOTXN

```
Transaction: VOTXN → press Enter



Text Object list → scroll to find EKKO and EKPO:

  EKKO   ← Purchase Order Header — click + click 'Text types' button

  EKPO   ← Purchase Order Item   — click + click 'Text types' button



Shows: Text ID, Description, Access Sequence, Refer/Duplicate flag

This controls: whether text is COPIED or REFERENCED when PO is created
```

Path 2 — Texts for Output Messages (What Gets Printed): SPRO

SPRO → Materials Management → Purchasing → Messages → Texts for Messages → Define Texts for Purchase Order ← as shown in SPRO screenshot Purpose: defines which Text IDs are included in the PO output message (print/email to vendor). This is NOT about copy/reference rules — it is about which texts appear on the PO form.

Configuration	Transaction / SPRO Path	Controls	Used For REF Fields?
Text Determination	VOTXN → EKKO / EKPO → Text types	Copy or Reference rules per Text ID. Access Sequence priority.	YES — this is where REF_TEXT_OBJECT source is defined
Texts for Output Messages	SPRO → MM → Purchasing → Messages → Texts for Messages → Define Texts for PO	Which Text IDs are printed on PO output form sent to vendor	NO — output format only, no impact on REF fields

PO Header Text Determination (EKKO)

```
Text Object:  EKKO

Text Schema:  0001 (or configured schema)



Text ID   Description          Access Seq   Refer/Duplicate   Effect

F01       Header note          —            unchecked         COPY — user types own text

F02       Delivery text        001          checked           REFERENCE from Info Rec / Vendor

F03       Pricing types        —            unchecked         COPY

F04       Deadlines            —            unchecked         COPY
```

```
F05       Terms of delivery    002          checked          REFERENCE from Vendor Master


Access Sequence 001 search order for EKKO F02:
  Priority 1: Purchasing Info Record (EINE/EINA) — vendor+material specific
  Priority 2: Vendor Master (LFA1/LFM1) — vendor general delivery text
  Priority 3: Nothing found → F02 stays empty in PO
```

PO Item Text Determination (EKPO)

```
Text Object:  EKPO
Text Schema:  0002 (or configured schema)


Text ID   Description          Access Seq   Refer/Duplicate   Effect
F01       Item note            —            unchecked         COPY — user types per item
F02       Delivery text        001          checked           REFERENCE from Info Record
F09       GR text              003          checked           REFERENCE from Info Record
F11       Info rec PO text     002          checked           REFERENCE from Info Record
F12       Material PO text     004          checked           REFERENCE from Material Master


Access Sequence 003 search order for EKPO F09 (GR text):
  Priority 1: Purchasing Info Record (EINE/EINA) — item specific
  Priority 2: Nothing found → F09 stays empty
```

The 6 Reference Scenarios for PO

1	**PO from Vendor Master text**	*Chain: EKKO → LFA1*
When	Vendor master LFA1/LFM1 has delivery instructions. PO created for this vendor.	
Source	**LFA1/LFM1**	
Result	Vendor delivery note F02 appears in PO — referenced, not copied. If vendor updates delivery hours, ALL POs referencing it auto-update.	

2	**PO Item text from Purchasing Info Record**	*Chain: EKPO → EINE*
When	Info record (EINE/EINA) for vendor+material has a GR inspection note.	
Source	**EINE/EINA**	
Result	PO item F09 (GR text) references the info record. Goods receiver sees the inspection instruction from the info record.	

3	PO Item text from Material Master	*Chain: EKPO → MATERIAL*
When	Material master has a standard purchase order text for the material.	
Source	**MATERIAL**	
Result	PO item F01 (General note) references material master. Appears on PO output to vendor without storing per PO.	

4	PO from Purchase Requisition	*Chain: EKKO/EKPO → EBAN*
When	PO created with reference to PR 1000012345 which had requestor notes.	
Source	**BANF/EBAN**	
Result	PO text may reference PR text depending on VOTXN config. Requestor's urgency note appears on PO without copying.	

5	PO from RFQ/Quotation	*Chain: EKKO → EKKO*
When	PO created with reference to RFQ 6000001234. Vendor terms noted in RFQ.	
Source	**EKKO (RFQ)**	
Result	PO header text references RFQ header text. Vendor's quoted delivery terms appear in PO — reference chain EKKO→EKKO.	

6	User edits referenced text in ME22N	*Chain: EKKO: REF→COPY*
When	User opens PO, sees referenced text, modifies it.	
Source	**—**	
Result	Reference BROKEN — becomes own copy. REF fields cleared. TEXT_CONTENT set to new value. Changes to source no longer affect this PO.	

Reference vs Copy — Decision Table

Trigger	Reference Created	Copy Created
VOTXN: Refer/Duplicate = checked	✓	
VOTXN: Refer/Duplicate = unchecked		✓
User types NEW text in ME21N/ME22N		✓ Own text
User EDITS existing reference in ME22N		✓ Reference broken → copy
No source text found in access sequence		Neither — text empty

Migration Impact for PO EKKO/EKPO: Records with TEXT_CONTENT = '' and REF fields populated = REFERENCES During SDM: REF fields are migrated as-is (pointer preserved in new table) Cross-object references (EKKO→LFA1, EKPO→MATERIAL) cannot be resolved in CDS joins because LFA1/MATERIAL text CDS views do not exist yet. Same-object references (EKKO→EKKO e.g. PO from RFQ) can be resolved with 1-level self-join.